

Part F — Written Analysis

Sorting Algorithms on Real-World Big Data

Algorithm Performance on Random Data

Benchmarking across sample sizes from 100 to 10,000 rows on the `ARR_DELAY` column revealed clear performance differences between the three algorithms. On random-ordered input, bubble sort was consistently the slowest, taking 5.80 seconds at $n=10,000$. Selection sort and insertion sort were nearly identical at that size, clocking in at 2.15 and 2.13 seconds respectively, making them roughly 2.7x faster than bubble sort on random data.

All three algorithms exhibit the expected $O(n^2)$ growth pattern. At $n=1,000$, bubble sort took 0.052 seconds. At $n=10,000$ — a 10x increase in input size — it took 5.80 seconds, a roughly 112x increase in runtime. This is consistent with $O(n^2)$: doubling n quadruples the time, and a 10x increase produces approximately a 100x increase. The same pattern holds for selection sort and insertion sort, confirming that all three algorithms behave as theory predicts in practice.

Early testing at small sizes ($n=100$, $n=500$) showed all three algorithms completing in under 15 milliseconds, which was fast enough to seem harmless. It was only when scaling past $n=2,500$ that the quadratic growth became visually apparent in the charts and practically painful to wait for.

Insertion Sort Across Orderings

Insertion sort showed the most dramatic variation across input orderings, which is exactly what its complexity analysis predicts. On sorted input, insertion sort took just 0.001 seconds at $n=10,000$ — effectively $O(n)$ because the inner while loop never executes when each element is already in place. On random input at the same size it took 2.13 seconds, and on reversed input it took 4.31 seconds — its true worst case, where every element must be shifted all the way to the front of the sorted portion.

This behaviour matches the theoretical best case of $O(n)$ for sorted input and worst case of $O(n^2)$ for reverse-sorted input. The gap between best and worst case at $n=10,000$ is a factor of over 3,000x, which is a striking real-world demonstration of why input ordering matters so much for insertion sort. Notably, bubble sort showed similar sensitivity: 0.0008 seconds on sorted input versus 8.12 seconds reversed at $n=10,000$, due to its early-exit optimisation on sorted data.

Numeric vs Text Column

At $n=1,000$ on random-ordered data, sorting `OP_CARRIER` (text) took longer than `ARR_DELAY` (numeric) for all three algorithms. Bubble sort took 0.052 seconds on numeric and 0.058 seconds on text. Selection sort took 0.022 seconds numeric versus 0.050 seconds text — a 2.3x difference. Insertion sort showed 0.021 seconds numeric versus 0.034 seconds text.

String comparison is more expensive than numeric comparison because Python cannot compare two strings with a single CPU instruction the way it can with floats. String comparison is character-by-character, and the cost scales with string length. Additionally,

`OP_CARRIER` codes in this dataset are short (2 characters), so the overhead is modest — on longer strings such as airport names or full carrier names, the difference would be far more pronounced.

Extrapolation to Full Scale

The filtered dataset from 2018.csv contains 7,213,446 rows. Using the constant c estimated from $n=10,000$ on random input:

- Bubble sort: $c = 5.80 / (10,000^2) = 5.80 \times 10^{-8} \rightarrow$ estimated full sort: **approximately 8.5 years**
- Selection sort: $c = 2.15 / (10,000^2) = 2.15 \times 10^{-8} \rightarrow$ estimated full sort: **approximately 3.2 years**
- Insertion sort: $c = 2.13 / (10,000^2) = 2.13 \times 10^{-8} \rightarrow$ estimated full sort: **approximately 3.1 years**

By contrast, Timsort ($O(n \log n)$) estimated using the same machine speed baseline would sort the full dataset in a matter of seconds. The speedup factor is on the order of hundreds of millions, making $O(n^2)$ algorithms completely impractical at this scale.

Based on these results, the crossover point where $O(n^2)$ algorithms become unacceptably slow in practice is somewhere around $n=5,000$ to $n=10,000$, where runtimes start exceeding several seconds. For anything beyond that, switching to an $O(n \log n)$ algorithm such as Timsort, merge sort, or heap sort is strongly recommended.

When Simple Algorithms Are Acceptable

Despite their poor scaling, bubble sort, selection sort, and insertion sort have legitimate real-world use cases under the right conditions. Insertion sort in particular is an excellent choice for small arrays ($n < 50$) due to its low constant factor and cache-friendly memory access pattern — which is exactly why Timsort uses insertion sort internally on small subarrays.

For nearly-sorted data, insertion sort approaches $O(n)$ and can outperform more complex algorithms that carry higher overhead. If the dataset is known to be almost in order with only a few elements out of place, insertion sort is a practical and simple choice. Selection sort, while never achieving better than $O(n^2)$, is useful when the cost of writing to memory is high, since it makes at most n swaps regardless of input ordering.

In summary: these algorithms are acceptable when the dataset is small (under a few thousand rows), when memory writes are expensive, or when the input is known to be nearly sorted. On large, unordered datasets like the airline delay data used here, they are entirely impractical and should be replaced with $O(n \log n)$ alternatives.